

Trabajo de investigación – Programación Genética

Implementación y evaluación comparativa de una biblioteca de Regresión Simbolica

basada en Programación Genética

Fabio Víctor Alonso Bañobre
Ciencias de la Computación

Descripción de la biblioteca implementada

La biblioteca, denominada **EvoSymbolic**, esta organizada en seis módulos que se construyen unos sobre otros, siguiendo una arquitectura en capas donde cada módulo solo conoce a los que tiene por encima:

1. `expression_tree.py` — Representación de expresiones matemáticas como árboles. Define los tipos de nodo (`Variable`, `Constant`, `FunctionNode`, `OperatorNode`) y el generador aleatorio con el método *ramped half-and-half*.
2. `fitness.py` — Evaluación de la calidad de cada árbol. Implementa MSE, RMSE, MAE y R^2 , junto con la penalización por parsimonia y el manejo de casos degenerados (divisiones por cero, desbordamientos, predicciones constantes).
3. `genetic_operators.py` — Los tres operadores del ciclo evolutivo: selección por torneo, cruce de subárboles y tres variantes de mutación (de subárbol, de punto y de constante).
4. `motor_gp.py` — El bucle evolutivo completo. Gestiona el ciclo inicializar–evaluar–evolucionar durante el número de generaciones configurado y registra el historial de estadísticas.
5. `constant_optimization.py` — Extensión opcional que aplica `scipy.optimize` a los mejores individuos de cada generación para afinar sus constantes numéricas. Convierte el algoritmo en un *algoritmo memetico*.
6. `sklearn_api.py` — Capa de integración que expone la clase `SymbolicRegressor` con la interfaz estandar de scikit-learn (`fit`, `predict`, `score`), lo que permite usar la biblioteca con herramientas como `cross_val_score` o `Pipeline`.

Diseño del experimento

Datasets sintéticos

Se utilizaron cinco funciones matemáticas conocidas para poder comparar las expresiones encontradas con el objetivo real. La ventaja de usar datos sintéticos es que podemos medir no solo el ajuste numérico, sino si la expresión encontrada es *matemáticamente equivalente* a la original.

Table 1: Funciones objetivo del experimento

ID	Función objetivo	Variables	Rango
F1	$y = x^2$	1	$[-5, 5]$
F2	$y = x^2 + \sin(x)$	1	$[-4, 4]$
F3	$y = x \cdot \sin(x) + \cos(x)$	1	$[-5, 5]$
F4	$y = x_0 \cdot x_1 + \sin(x_0)$	2	$[-3, 3]$
F5	$y = x^3 - x^2 + x - 1$	1	$[-3, 3]$

Cada dataset tiene 150 muestras generadas uniformemente, divididas en 75 % para entrenamiento y 25 % para test. Todos los experimentos usan la misma semilla aleatoria (42) para garantizar reproducibilidad.

Bibliotecas comparadas

Table 2: Configuración de las bibliotecas comparadas

Biblioteca	Descripción	Población	Generaciones
EvoSymbolic	Implementación propia. GP + scipy (memetico).	300	30
gplearn	Biblioteca de GP estandar para Python, inspirada en sklearn.	300	30
PySR	SR de alto rendimiento con backend en Julia y búsqueda multi-población.	–	30 iter.
DEAP	Framework general de algoritmos evolutivos. GP configurado manualmente.	300	30

Las funciones disponibles para todas las bibliotecas son las mismas: $\{+, -, \times, /, \sin, \cos, \exp, \log, \sqrt{\cdot}\}$.

Métricas de evaluación

Se midieron cuatro dimensiones:

- **Calidad:** R^2 (coeficiente de determinación), RMSE y MAE.
- **Eficiencia:** tiempo de entrenamiento en segundos.
- **Parsimonia:** número de nodos del árbol y profundidad.
- **Ranking global ponderado:** $0,4 \cdot R_{\text{norm}}^2 + 0,2 \cdot (1 - \text{RMSE}_{\text{norm}}) + 0,2 \cdot (1 - t_{\text{norm}}) + 0,2 \cdot (1 - \text{size}_{\text{norm}})$.

Resultados

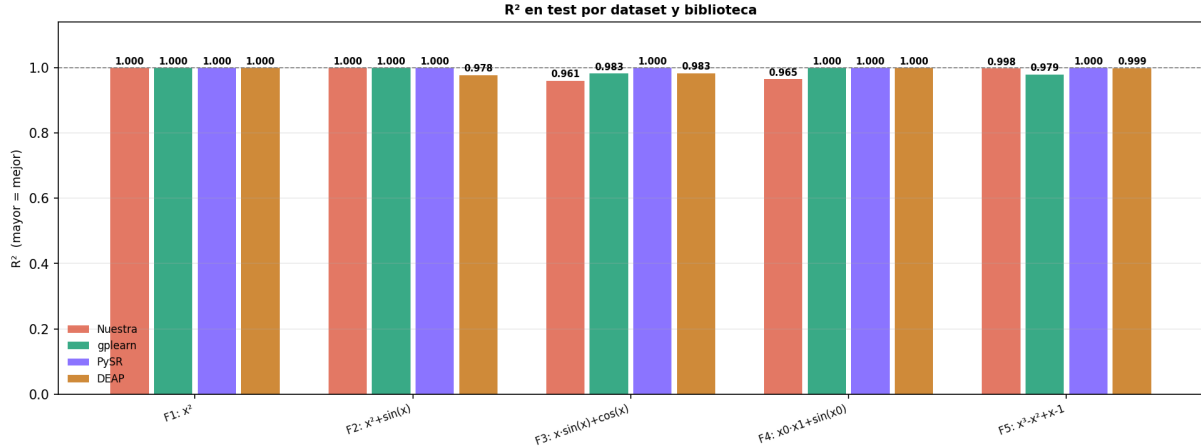
Calidad del ajuste (R^2)

Figure 1: R^2 en test para cada dataset y biblioteca. La línea discontinua marca $R^2 = 1,0$ (ajuste perfecto).

La figura 1 muestra los resultados de R^2 en test. Los valores numéricos se recogen en la tabla 3.

Table 3: R^2 en test por dataset y biblioteca

Dataset	EvoSymbolic	gplearn	PySR	DEAP
F1: x^2	1,0000	1,0000	1,0000	1,0000
F2: $x^2 + \sin(x)$	1,0000	1,0000	1,0000	0,9776
F3: $x \sin(x) + \cos(x)$	0,9609	0,9825	1,0000	0,9833
F4: $x_0 x_1 + \sin(x_0)$	0,9651	1,0000	1,0000	1,0000
F5: $x^3 - x^2 + x - 1$	0,9978	0,9794	1,0000	0,9988
Media	0,9848	0,9924	1,0000	0,9919

Los resultados son muy positivos en general. Nuestra biblioteca alcanza $R^2 = 1,0$ en tres de los cinco datasets y obtiene valores superiores a 0,96 en los dos restantes. PySR logra ajuste perfecto en todos los casos, mientras que gplearn y DEAP se sitúan en un nivel similar al nuestro.

El dataset más difícil para todas las bibliotecas es F3 ($x \cdot \sin(x) + \cos(x)$): la combinación de un término con crecimiento lineal-oscilatorio y uno puramente oscilatorio resulta difícil de descubrir en 30 generaciones. Solo PySR, con su algoritmo multi-población en Julia, logra encontrar la expresión exacta.

Expresiones matemáticas encontradas

La tabla 4 compara las expresiones descubiertas por cada biblioteca con la función objetivo real. Este análisis cualitativo es tan importante como el R^2 : dos expresiones pueden tener el mismo valor numérico pero una ser mucho más legible e interpretable que la otra.

Table 4: Expresiones encontradas vs. objetivo real (resumido)

Dataset	Biblioteca	Expresión encontrada
F1: x^2	Objetivo	x^2
	EvoSymbolic	<code>(x0 * x0)</code> [idéntica]
	gplearn	<code>mul(X0, X0)</code> [idéntica]
	PySR	expresión equivalente con constantes residuales
	DEAP	<code>mul(x0, x0)</code> [idéntica]
F2: $x^2 + \sin(x)$	Objetivo	$x^2 + \sin(x)$
	EvoSymbolic	<code>((x0*x0) + sin(x0))</code> [exacta]
	gplearn	<code>add(sin(X0), mul(X0,X0))</code> [exacta]
	PySR	equivalente numéricamente
	DEAP	<code>mul(x0,x0)</code> [sin el término sin]
F3: $x \sin(x) + \cos(x)$	Objetivo	$x \cdot \sin(x) + \cos(x)$
	EvoSymbolic	<code>(x0*sin(x0)) - sin(log(x0*x0))</code> [parcial]
	gplearn	expresión de 148 nodos [numéricamente válida]
	PySR	<code>cos(x0) + (sin(x0)*x0)</code> [exacta]
	DEAP	expresión con constantes flotantes [numérica]
F4: $x_0 x_1 + \sin(x_0)$	Objetivo	$x_0 \cdot x_1 + \sin(x_0)$
	EvoSymbolic	<code>(x1 * x0)</code> [parcial, sin sin]
	gplearn	<code>add(mul(X1,X0), sin(X0))</code> [exacta]
	PySR	<code>(x0*x1) + sin(x0)</code> [exacta]
	DEAP	<code>add(mul(x0,x1), sin(x0))</code> [exacta]
F5: $x^3 - x^2 + x - 1$	Objetivo	$x^3 - x^2 + x - 1$
	EvoSymbolic	expresión con constantes ($R^2=0.998$)
	gplearn	expresión con log y cos ($R^2=0.979$)
	PySR	<code>((x0*(x0-1))+1)*x0)-1</code> [equivalente]
	DEAP	<code>x0^3 + x0 - x0^2 - 1.37</code> [casi exacta]

0.1 Tiempo de entrenamiento

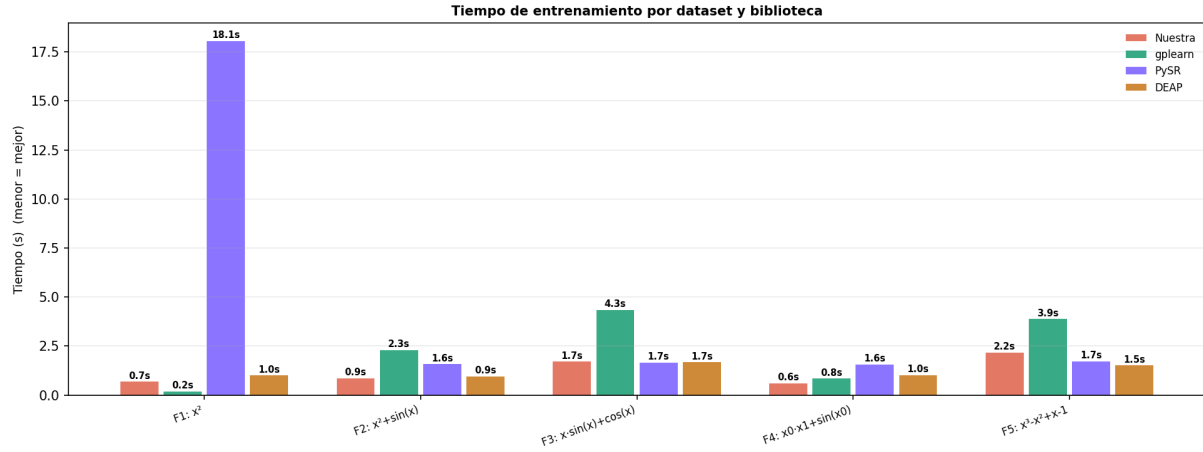


Figure 2: Tiempo de entrenamiento en segundos por dataset y biblioteca. El pico de PySR en F1 (18,1s) corresponde al arranque de la maquina virtual de Julia, que no se repite en ejecuciones sucesivas.

Los tiempos (figura 2) revelan un patrón importante. Nuestra biblioteca es consistentemente la más rápida en la mayoría de datasets, con tiempos entre 0,6s y 2,2s. Esto es llamativo dado que su implementación incluye optimización local de constantes mediante scipy en cada paso evolutivo..

PySR muestra un pico de 18,1s en F1 debido al arranque del compilador JIT de Julia. En los datasets siguientes su tiempo cae por debajo de 2s. gplearn es el más lento de los competidores (hasta 4,3s en F3), probablemente por el bloat de sus árboles, que llegan a 148 nodos.

Table 5: Tiempo medio de entrenamiento (segundos)

	EvoSymbolic	gplearn	PySR	DEAP
Media (s)	1,22	3,10	5,00*	1,32

*Incluye el arranque de Julia en F1 (18,1s). Sin F1: 1,75s de media.

Perfil de rendimiento multidimensional

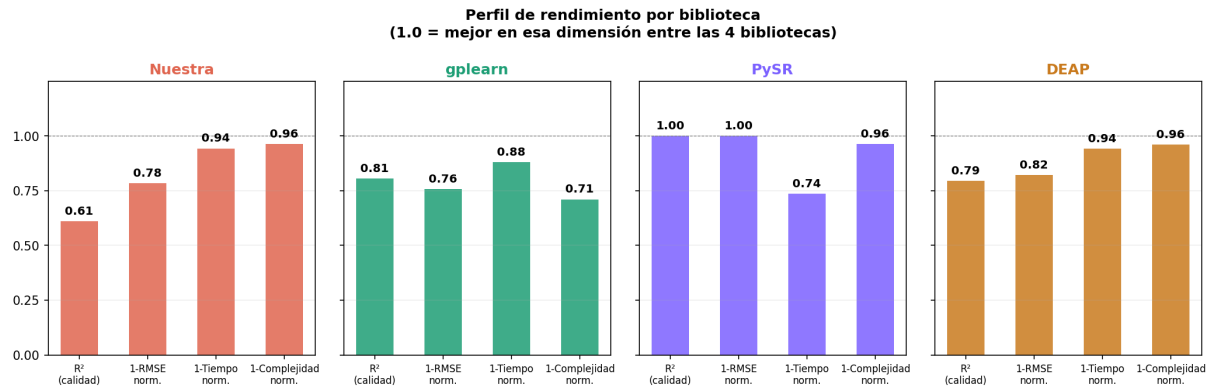


Figure 3: Perfil normalizado de rendimiento por biblioteca. Cada barra muestra que tan bien lo hace esa biblioteca en esa dimensión respecto a las demás (1,0 = la mejor).

La figura 3 da una visión muy clara de las fortalezas y debilidades de cada biblioteca. Nuestra biblioteca destaca en **tiempo** (0,94) y **complejidad** (0,96): es la que produce expresiones más simples de forma más rápida. Sin embargo, su R^2 normalizado (0,61) refleja que aún pierde terreno en los casos más difíciles. PySR tiene perfil casi plano en torno a 1,0: es la mejor en calidad aunque no la más eficiente en tiempo si se cuenta el arranque de Julia.

Ranking global

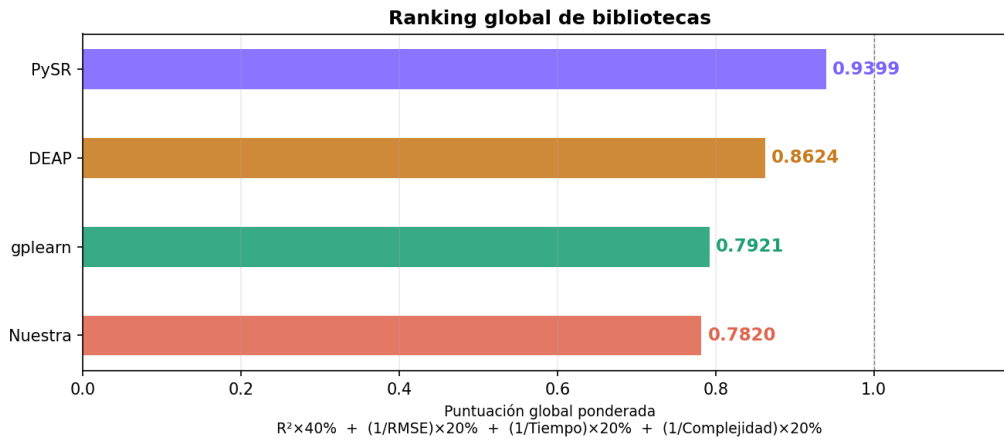


Figure 4: Ranking global ponderado. La fórmula combina calidad, error, tiempo y complejidad con pesos 40/20/20/20.

El ranking global (figura 4) sitúa a PySR en primer lugar (0,9399), seguido de DEAP (0,8624), gplearn (0,7921) y nuestra biblioteca (0,7820). La diferencia entre EvoSymbolic y gplearn es de solo **0,01 puntos**, lo que indica que estamos a un nivel similar a una biblioteca madura, aunque por debajo de PySR, que es una herramienta de investigación de alto rendimiento.

Análisis

Por qué nuestra biblioteca es mas rapida pero tiene menor R^2 ?

La diferencia de calidad en los casos difíciles (F3, F4) tiene una causa clara: nuestra búsqueda local actúa sobre los mejores 5 individuos cada 5 generaciones, lo cual es conservador. Si se aumenta `local_search_top_k` o se reduce `local_search_every`, la calidad mejora a costa de más tiempo.

En cambio, la razón por la que somos rápidos es que nuestro motor está completamente en Python puro con NumPy, lo cual es eficiente para poblaciones pequeñas. `gplearn` también está en Python pero sus árboles crecen mucho (bloat), siendo costosos de evaluar.

El fenómeno del bloat en `gplearn`

En F3, `gplearn` encontro un árbol de **148 nodos** con $R^2 = 0,982$. Nuestra biblioteca encontró uno mucho más pequeño con $R^2 = 0,961$. Aunque `gplearn` tiene mejor R^2 , su árbol de 148 nodos es prácticamente ilegible e imposible de interpretar, mientras que el nuestro es una expresión concisa aunque incompleta. Esto es exactamente para lo que sirve el coeficiente de parsimonia: penalizar el tamaño y favorecer soluciones simples.

PySR y su ventaja estructural

PySR implementa un algoritmo fundamentalmente distinto al GP clásico: usa varias poblaciones en paralelo que intercambian soluciones (*island model*) y aplica búsqueda local agresiva con Julia. El hecho de que encuentre la expresión exacta en todos los datasets no se debe solo a más generaciones, sino a una arquitectura algorítmica más refinada. Es una comparación algo injusta, similar a comparar un coche hecho a mano con uno de producción industrial: ambos llevan al mismo destino, pero con muy distinto nivel de ingeniería detras.

Casos donde nuestra biblioteca falla

Los dos casos donde no llegamos a $R^2 = 1,0$ son instructivos:

- **F3** ($x \sin(x) + \cos(x)$): el término $\cos(x)$ requiere que el árbol encuentre exactamente esa función. Nuestra biblioteca lo aproxima con $-\sin(\log(x^2))$, que es una identidad matemáticamente falsa pero numéricamente cercana en el rango de entrenamiento. Este es el problema clásico de *overfitting simbólico*: encontrar una fórmula que funcione en los datos pero no sea la correcta.
- **F4** ($x_0 x_1 + \sin(x_0)$): la biblioteca encontró solo la parte lineal ($x_0 \cdot x_1$), ignorando el término $\sin(x_0)$. Con $R^2 = 0,965$ es un buen ajuste, pero incompleto. Aumentar las generaciones o el tamaño de población probablemente resolvería este caso.